

## Tarefa 13: Escalabilidade com Afinidade de Threads

Navier-Stokes com OMP\_PROC\_BIND e OMP\_PLACES em um nó do NPAD

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Setembro de 2026

### 1. Objetivo

Avaliar como a escalabilidade do código de Navier-Stokes muda ao utilizar os diversos tipos de afinidade de threads suportados pelo sistema operacional e pelo OpenMP, no mesmo tipo de nó de computação do NPAD utilizado na Tarefa 12.

### 2. Ambiente de Execução

| Item        | Valor                                                             |
|-------------|-------------------------------------------------------------------|
| Máquina     | Supercomputador NPAD/UFRN, partição amd-512, nó exclusivo         |
| Nó          | r2n09 (o r2n21 da Tarefa 12 estava em manutenção; mesmo hardware) |
| Processador | 2 × AMD EPYC 7713 (Milan), 128 núcleos                            |
| NUMA        | 8 domínios de 16 núcleos (0–15 no domínio 0, 16–31 no 1, ...)     |
| Compilador  | GCC 14.2.0, -O3 -march=native -fopenmp                            |
| Job         | SLURM 2127798 (conferência de afinidade: 2127837)                 |

### 3. Metodologia

O código é a versão final (v4) da Tarefa 12: difusão viscosa com stencil de 5 pontos, inicialização paralela (first touch) e uma só região paralela envolvendo o laço do tempo. O cálculo não foi alterado; a afinidade é escolhida apenas por variáveis de ambiente, como no slide:

- OMP\_PROC\_BIND=false: sem afinidade; o **sistema operacional** decide onde cada thread roda e pode migrá-la (o OMP\_PLACES é ignorado).
- OMP\_PROC\_BIND=true, master, close, spread com OMP\_PLACES=cores: cada thread fica presa a um núcleo físico, escolhido pela política do **OpenMP**.

O problema é o da escalabilidade forte da Tarefa 12: grade 4096×4096 (duas grades de 134 MB) e 100 passos, com 1, 2, 4, ..., 128 threads. Cada caso rodou 3 vezes e foi usado o menor tempo. O speedup é calculado em relação ao melhor tempo com 1 thread (1,16 s).

## 4. Resultados

Tempo em segundos e, entre parênteses, o speedup:

| Threads | false (SO)    | true          | master         | close         | spread        |
|---------|---------------|---------------|----------------|---------------|---------------|
| 1       | 1,163 (1,0×)  | 1,158 (1,0×)  | 1,163 (1,0×)   | 1,170 (1,0×)  | 1,160 (1,0×)  |
| 2       | 0,823 (1,4×)  | 0,926 (1,3×)  | 1,074 (1,1×)   | 0,926 (1,3×)  | 0,574 (2,0×)  |
| 4       | 0,342 (3,4×)  | 0,941 (1,2×)  | 1,321 (0,9×)   | 0,942 (1,2×)  | 0,275 (4,2×)  |
| 8       | 0,122 (9,5×)  | 1,013 (1,1×)  | 2,159 (0,5×)   | 1,012 (1,1×)  | 0,119 (9,7×)  |
| 16      | 0,061 (19,1×) | 0,807 (1,4×)  | 2,872 (0,4×)   | 0,811 (1,4×)  | 0,060 (19,3×) |
| 32      | 0,036 (32,0×) | 0,289 (4,0×)  | 6,143 (0,2×)   | 0,291 (4,0×)  | 0,034 (33,6×) |
| 64      | 0,030 (38,9×) | 0,053 (22,0×) | 12,652 (0,09×) | 0,052 (22,4×) | 0,025 (45,8×) |
| 128     | 0,037 (31,4×) | 0,030 (39,0×) | 25,984 (0,04×) | 0,029 (39,5×) | 0,018 (63,8×) |

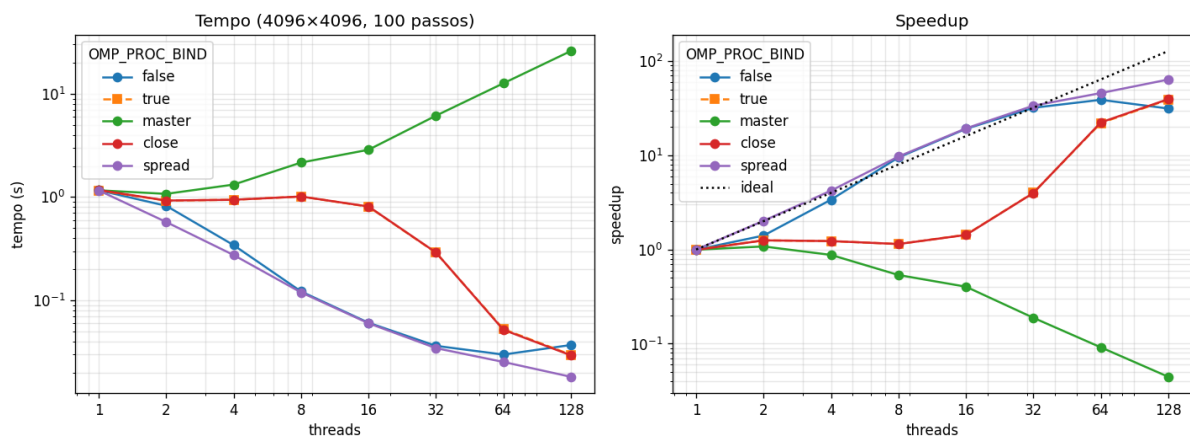


Figura 1 — Tempo (esquerda) e speedup (direita) para cada política de OMP\_PROC\_BIND. A curva de true (tracejada) coincide com a de close.

## 5. Análise

**spread é a melhor política em todos os casos** (19,3× com 16 threads e 63,8× com 128). Com OMP\_PLACES=cores as threads ficam espalhadas pelo nó: com 8 threads há uma em cada domínio NUMA. Como o stencil é limitado pela banda de memória, cada thread usa um controlador de memória diferente, e o first touch deixa as linhas de cada thread na memória do seu domínio. Com 8 e 16 threads o speedup passa do ideal, porque as threads espalhadas somam o cache L3 de vários CCDs e as grades passam a caber quase inteiras em cache.

**close quase não escala até 16 threads** (1,2× a 1,4×): as threads ficam em núcleos vizinhos (0, 1, 2, ...), ou seja, todas no domínio NUMA 0, disputando a banda de memória de um só domínio. O ganho só aparece quando as threads chegam a outros domínios: 4,0× com 32 (2 domínios) e 22× com 64 (um socket inteiro). Com 128 threads, close e spread colocam a thread  $i$  no núcleo  $i$  (conferido com OMP\_DISPLAY\_AFFINITY no job 2127837); a diferença restante (0,029 s contra 0,018 s) vem da variação entre execuções tão curtas: no job de conferência, close variou de 0,016 a 0,031 s.

**true dá os mesmos tempos que close:** no GCC (libgomp), true usa a mesma distribuição de close.

**master piora com mais threads:** todas as threads ficam no núcleo da thread 0 e dividem um único núcleo, e cada thread a mais só acrescenta troca de contexto e sincronização (26,0 s com 128 threads, 22× mais lento que com 1).

**false (sistema operacional) fica perto de spread até 32 threads,** porque o escalonador do Linux distribui as threads pelos núcleos livres. Com 64 e 128 threads fica pior (38,9× e 31,4×, contra 45,8× e

63,8× de spread): sem afinidade, as threads podem migrar e se afastar da memória onde fizeram o first touch, e o SO também pode colocar duas threads nos dois hyperthreads de um mesmo núcleo (o nó expõe 256 CPUs lógicas).

Na Tarefa 12, que usou spread + cores, a mesma versão chegou a 0,011 s (106,9×) com 128 threads no r2n21. Aqui foram 0,018 s no r2n09 e 0,013–0,015 s no job de conferência: com 128 threads cada execução dura só ~15 ms e varia bastante entre execuções e entre nós.

## 6. Conclusão

Com o mesmo código e o mesmo tipo de nó, só a afinidade muda o speedup com 16 threads de 0,4× (master) a 1,4× (close) e 19,3× (spread). Como o Navier-Stokes é limitado pela banda de memória, a melhor escolha no nó NUMA do NPAD é espalhar as threads (OMP\_PROC\_BIND=spread, OMP\_PLACES=cores): todos os controladores de memória são usados desde poucas threads e o first touch continua valendo. Deixar o sistema operacional decidir funciona bem até 32 threads, mas perde desempenho com o nó cheio, e close/true só se aproximam de spread quando todos os núcleos estão em uso.

## 7. Código-Fonte

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  #define R 0.01 /* nu*dt/dx^2 */
6
7  int main(int argc, char **argv) {
8      if (argc != 3) return 1;
9      int n = atoi(argv[1]), passos = atoi(argv[2]);
10     double *u = malloc(sizeof(double) * n * n);
11     double *v = malloc(sizeof(double) * n * n);
12
13     /* first touch: cada thread inicializa as linhas que vai calcular */
14     #pragma omp parallel for schedule(static)
15     for (int i = 0; i < n; i++)
16         for (int j = 0; j < n; j++) {
17             u[i * n + j] = (i == n / 2 && j == n / 2) ? 1.0 : 0.0;
18             v[i * n + j] = 0.0;
19         }
20
21     double t0 = omp_get_wtime();
22     #pragma omp parallel
23     for (int t = 0; t < passos; t++) {
24         double *a = (t % 2 == 0) ? u : v;
25         double *b = (t % 2 == 0) ? v : u;
26         #pragma omp for schedule(static)
27         for (int i = 1; i < n - 1; i++)
28             for (int j = 1; j < n - 1; j++)
29                 b[i * n + j] = a[i * n + j] + R * (a[(i + 1) * n + j] + a[(i - 1) * n + j]
30                 + a[i * n + j + 1] + a[i * n + j - 1] - 4 * a[i * n + j]);
31     }
32     printf("%d,%f\n", omp_get_max_threads(), omp_get_wtime() - t0);
33
34     free(u);
35     free(v);
36     return 0;
37 }
38
```

Figura 2 — fluid.c.